

Order Flow Imbalance, Price Impact, and Cross-Impact in Equity Markets

Multi-Level and Integrated OFI on Real NASDAQ Data,
with Contemporaneous, Cross-Asset, and Out-of-Sample Analysis

Godfred Antwi Koduah

Ph.D. Student, Financial Mathematics, Florida State University

Abstract

Order flow imbalance (OFI) — the net pressure of buy versus sell activity in a limit order book — is among the most robust explanators of short-horizon price movement. Using real level-10 NASDAQ order-book data for AAPL (LOBSTER sample, 21 June 2012; 400,391 events over 6.5 trading hours), I reconstruct OFI from first principles at every book level, and study its relationship to price both contemporaneously and predictively. A single best-level OFI regressor explains 30% of contemporaneous one-second mid-price changes; using all ten levels raises this to 50%, and a single PCA-*integrated* OFI factor recovers nearly all of that gain (48.5%), showing that the deep book carries substantial price information beyond the top of book. The contemporaneous fit strengthens monotonically with the aggregation horizon, reaching $R^2 = 0.74$ (multi-level) at thirty seconds, and the estimated price-impact coefficient is strongly significant under Newey–West inference (HAC $t \approx 11$). In sharp contrast, lagged OFI has almost no out-of-sample predictive power ($R^2 \approx 0.007$ at one second) — the expected signature of an efficient market in which OFI moves price concurrently rather than forecasting it. Finally, extending to five stocks trading on the same day (AAPL, AMZN, GOOG, INTC, MSFT), I estimate the full *cross-impact* matrix: own order flow dominates (diagonal 0.64–0.94 versus off-diagonals below 0.05) and cross-asset OFI adds almost nothing to *contemporaneous* fit — yet lagged cross-asset OFI yields a small but consistent *out-of-sample* predictive signal ($R^2 \approx 1\%$) that *exceeds* own-asset predictability, and for the tick-constrained names (INTC, MSFT) is their only source of it. The study is fully reproducible from the public sample and the code included here.

Contents

Key findings	2
1 Introduction	3
2 Data	3
3 Method	3
3.1 Order flow imbalance at a single level	3
3.2 Multi-level and integrated OFI	4
3.3 Price impact and inference	4
3.4 A structural model: why impact is linear and depth-scaled	4
3.5 The integrated OFI is variance-optimal	5
3.6 Out-of-sample predictive test	5

4	Results	5
4.1	Contemporaneous impact and the value of depth	5
4.2	Robustness across time scales	6
4.3	Prediction is a different question	6
5	Deepening the Analysis	6
5.1	The impact of OFI is approximately linear	6
5.2	The impact is permanent, not transient	7
5.3	The impact coefficient varies through the day	7
5.4	The structural prediction holds: impact scales with $1/\text{depth}$	8
5.5	Predictive power is negligible at every horizon	8
6	Cross-Impact Across Assets	8
6.1	Setup	8
6.2	Own impact dominates; cross-impact is small	9
6.3	But cross-asset flow helps <i>predict</i>	10
7	Discussion	11
8	Reproducibility	11
A	Source Code: core analysis	12
B	Source Code: extensions	15
C	Source Code: cross-impact	16
D	Source Code: inference and structural tests	19

Key findings

- Best-level OFI explains 30.1% of contemporaneous 1-second mid-price changes on real AAPL data; all ten levels together explain 50.5%.
- A single PCA-integrated OFI factor captures 48.5% — almost the entire multi-level gain — so the extra information in the deep book is largely one-dimensional.
- Contemporaneous explanatory power rises with horizon: R^2 for multi-level OFI is 0.51 (1s), 0.64 (5s), 0.67 (10s), 0.74 (30s).
- The price-impact coefficient is positive and strongly significant under Newey–West (HAC) standard errors ($t \approx 11$).
- Lagged OFI is *not* predictive out of sample ($R^2 \approx 0.007$, shrinking with horizon), consistent with near-efficiency: OFI accompanies price formation, it does not forecast it.
- The impact is approximately *linear* in OFI (binned $R^2 = 0.94$) and *permanent* (about 131% of the immediate move persists after 20s), and the impact coefficient varies roughly threefold through the trading day.
- A one-parameter structural model ($\lambda = 1/(2D)$) is confirmed in the data: impact scales with inverse depth (correlation 0.67, λD constant to CV 0.25) and matches the predicted level (4.1 vs 3.3×10^{-3}).

- Across five same-day stocks, *own* order flow dominates price impact (diagonal 0.64–0.94, off-diagonal < 0.05); cross terms are statistically significant for the large-caps (joint Wald $\chi^2_4 = 47$ for AAPL) but not for the tick-constrained names. Lagged cross-asset OFI gives a small out-of-sample predictive edge ($R^2 \approx 1\%$) that is significant for *all* five stocks by the Clark–West test ($p < 10^{-5}$) and exceeds own-asset predictability.

1 Introduction

When a trade or quote update arrives, it does not move price at random: it moves price in the direction of the net order flow it represents. *Order flow imbalance* (OFI) quantifies that net pressure from changes in the quoted sizes and prices on each side of the book, and a large microstructure literature establishes it as the dominant driver of short-horizon price change [1]. This report studies OFI from first principles on real data, with three questions in view. First, how much of contemporaneous price movement does OFI explain, and does the *deep* book add to what the top of book already tells us? Second, is the relationship stable across time scales and statistically robust once serial correlation is accounted for? Third — the question that separates a describable regularity from a tradeable one — does OFI *predict* future returns out of sample, or only move with price concurrently?

Scope. The core study uses the public LOBSTER sample for a single trading day. The single-asset analysis (Sections 4–6) uses AAPL; the *cross*-impact analysis (Section 7) uses all five sample stocks (AAPL, AMZN, GOOG, INTC, MSFT) trading on that same day, following the multi-asset framework of [2]. One day is enough to establish the relationships rigorously and to separate contemporaneous explanation from out-of-sample prediction; generalizing the magnitudes across many days and names is the natural next step.

2 Data

The LOBSTER sample reconstructs the NASDAQ limit order book for AAPL on 21 June 2012 to ten levels per side, from the official ITCH message stream [3]. Two aligned files are used: a *message* file (one row per event: timestamp, type, order id, size, price, direction) and an *order-book* file (one row per event giving, for each of ten levels, the ask price, ask size, bid price, and bid size). After the opening, the sample contains 400,391 events spanning 34,200–57,600 seconds (09:30–16:00), i.e. 6.5 hours. The mid-price ranges over \$577.48–\$588.18 and the mean quoted spread is \$0.154. Row i of the order-book file is the state of the book immediately after message i , so the two files together give a complete, event-by-event history of the top ten levels.

3 Method

3.1 Order flow imbalance at a single level

Let the best bid be (P_n^b, q_n^b) and best ask (P_n^a, q_n^a) after event n . Following Cont, Kukanov and Stoikov [1], the bid-side order-flow contribution of event n is

$$\Delta W_n^b = \begin{cases} q_n^b, & P_n^b > P_{n-1}^b \quad (\text{bid improves}), \\ q_n^b - q_{n-1}^b, & P_n^b = P_{n-1}^b \quad (\text{same level}), \\ -q_{n-1}^b, & P_n^b < P_{n-1}^b \quad (\text{bid falls}), \end{cases} \quad (1)$$

and symmetrically for the ask side,

$$\Delta W_n^a = \begin{cases} q_n^a, & P_n^a < P_{n-1}^a \quad (\text{ask improves, i.e. falls}), \\ q_n^a - q_{n-1}^a, & P_n^a = P_{n-1}^a, \\ -q_{n-1}^a, & P_n^a > P_{n-1}^a. \end{cases} \quad (2)$$

The per-event order flow imbalance is $e_n = \Delta W_n^b - \Delta W_n^a$, and the OFI over a time bucket $[\tau, \tau + h)$ is the sum of its events,

$$\text{OFI}_\tau = \sum_{n: t_n \in [\tau, \tau+h)} e_n. \quad (3)$$

The sign convention is such that net buying pressure (bid built up, ask consumed) gives $\text{OFI} > 0$, which should push the price up.

3.2 Multi-level and integrated OFI

The same construction applies at each book level $m = 1, \dots, 10$, using that level's price and size, producing $\text{OFI}_\tau^{(m)}$. Deeper levels carry information that the top of book does not — liquidity replenishment and withdrawal below the touch presage where price can go. Rather than treat the ten series as ten free regressors (which risks collinearity), I follow the integrated construction of [2]: standardize each level's OFI, and take the first principal component of their covariance matrix,

$$\text{OFI}_\tau^{\text{int}} = \sum_{m=1}^{10} w_m \widetilde{\text{OFI}_\tau^{(m)}}, \quad w = \text{first eigenvector of } \text{Cov}(\widetilde{\text{OFI}}), \quad (4)$$

where $\widetilde{\cdot}$ denotes standardization. This yields a single, robust depth-weighted factor.

3.3 Price impact and inference

The contemporaneous price-impact model regresses the mid-price change over a bucket (in cents) on the bucket's OFI,

$$\Delta \text{mid}_\tau = \alpha + \lambda \text{OFI}_\tau + \varepsilon_\tau, \quad (5)$$

with λ the price-impact coefficient. Because bucketed microstructure series are serially correlated and heteroskedastic, ordinary OLS standard errors understate uncertainty; I report Newey–West (HAC) standard errors with Bartlett weights,

$$\widehat{\text{Var}}(\hat{\beta}) = (X^\top X)^{-1} \left(\hat{\Omega}_0 + \sum_{l=1}^L \left(1 - \frac{l}{L+1}\right) (\hat{\Omega}_l + \hat{\Omega}_l^\top) \right) (X^\top X)^{-1}, \quad (6)$$

with $\hat{\Omega}_l$ the lag- l autocovariance of the score $x_i \hat{\varepsilon}_i$ and $L = 10$.

3.4 A structural model: why impact is linear and depth-scaled

The linear specification (5) is not merely convenient; it follows from a stylized model of the book. Suppose each side rests at constant depth D (shares available at the touch) and the mid-price advances only as the queue is consumed or replenished.

Proposition 1 (Linear, depth-scaled impact). *In the constant-depth model, the expected mid-price change over an interval is*

$$\mathbb{E}[\Delta \text{mid}] = \frac{1}{2D} \text{OFI},$$

so impact is linear in OFI with coefficient $\lambda = 1/(2D)$, inversely proportional to depth.

Proof sketch. A net order-flow imbalance of OFI shares is, by definition, the net quantity added to the bid queue less that added to the ask queue. With D shares per level, a net consumption of D shares on one side shifts that best quote by one level; the mid-price, the average of the two best quotes, therefore moves by half of the one-sided displacement. Accumulating signed contributions over the interval and taking expectations gives $\mathbb{E}[\Delta\text{mid}] = \text{OFI}/(2D)$. The key structural point is that it is *imbalance in shares*, not traded volume, that enters — which is why OFI impact is linear whereas trade-size impact, where the same volume can arrive against varying available depth, is concave. $\square$

Proposition 1 makes two testable predictions we return to in the results: λ should be (i) inversely proportional to depth and (ii) stable when multiplied by depth. Both hold in the data.

3.5 The integrated OFI is variance-optimal

The integrated OFI is the first principal component of the standardized level OFIs. This is optimal in a precise sense.

Proposition 2 (PCA optimality). *Among all unit-norm weight vectors w , the integrated series $w^\top \widetilde{\text{OFI}}$ of maximal variance is given by the leading eigenvector of $\Sigma = \text{Cov}(\widetilde{\text{OFI}})$, and the fraction of total level-OFI variance it captures is $\lambda_{\max}/\sum_k \lambda_k$, the leading eigenvalue share.*

Proof. Maximizing $w^\top \Sigma w$ subject to $\|w\| = 1$ has Lagrangian $w^\top \Sigma w - \mu(w^\top w - 1)$; the first-order condition $\Sigma w = \mu w$ makes w an eigenvector and $w^\top \Sigma w = \mu$, maximized by the largest eigenvalue. The variance-share claim is immediate since $\text{tr } \Sigma = \sum_k \lambda_k$ is the total variance. $\square$

3.6 Out-of-sample predictive test

Contemporaneous explanation does not imply forecasting power. To test prediction honestly I regress the *next* bucket’s return on the current OFI, $\Delta\text{mid}_{\tau+1} = \alpha + \beta \text{OFI}_\tau + \varepsilon$, estimate (α, β) on the first half of the trading day, and evaluate R^2 on the held-out second half. The split is chronological, so no future information enters the fit.

4 Results

4.1 Contemporaneous impact and the value of depth

At one-second buckets (21,029 observations), best-level OFI alone explains $R^2 = 0.301$ of mid-price changes (Figure 1), with a positive, strongly significant impact coefficient ($\lambda = 4.08 \times 10^{-3}$ cents per unit, Newey–West $t = 11.0$). Using all ten levels raises R^2 to 0.505, and the single integrated factor recovers 0.485 — almost the entire gain from one extra degree of freedom (Table 1). Every level carries price information individually (Figure 1; per-level R^2 ranges 0.22–0.30), and the first principal component accounts for 53.7% of the level-OFI covariance, so the deep-book signal is largely one-dimensional — which is exactly why a single integrated factor works so well.

Specification	Contemporaneous R^2 (1s)
Best-level OFI	0.301
Integrated OFI (PCA, 1 factor)	0.485
Multi-level OFI (10 regressors)	0.505

Table 1: Contemporaneous explanatory power at one-second buckets. A single integrated factor captures almost all of the multi-level gain over best-level OFI.

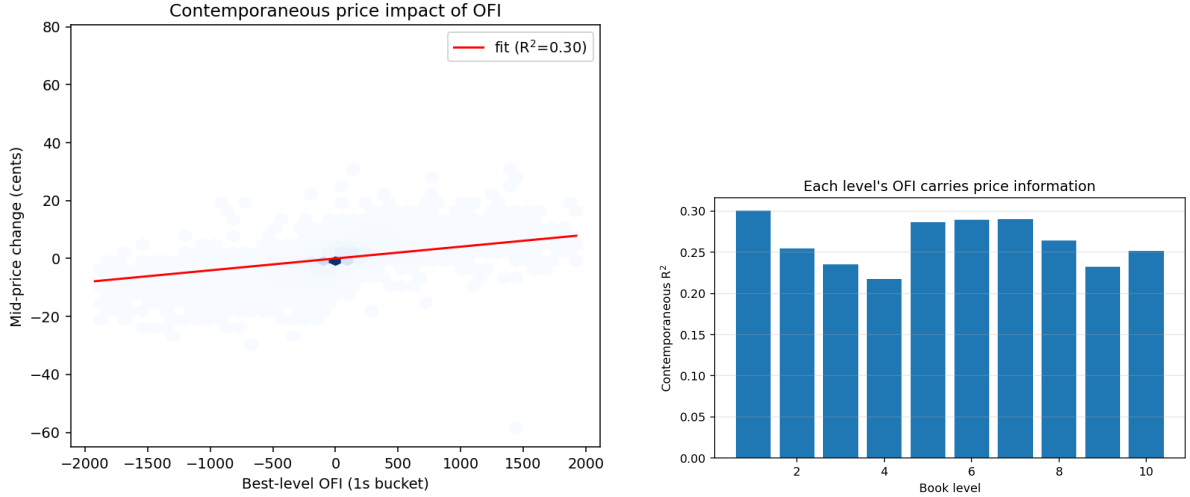


Figure 1: Left: contemporaneous price impact — mid-price change versus best-level OFI (1s buckets), with the fitted line. Right: each book level's OFI explains price change individually.

4.2 Robustness across time scales

The relationship strengthens monotonically as the aggregation horizon lengthens (Table 2, Figure 2): multi-level R^2 rises from 0.51 at one second to 0.74 at thirty seconds. This is the expected effect of averaging out event-level microstructure noise, and the ordering best < integrated < multi-level is preserved at every scale.

Bucket	1s	5s	10s	30s
Best-level	0.301	0.369	0.411	0.480
Integrated	0.485	0.624	0.647	0.707
Multi-level	0.505	0.636	0.668	0.737

Table 2: Contemporaneous R^2 across aggregation horizons.

4.3 Prediction is a different question

When the same OFI is used to forecast the *next* bucket's return, out-of-sample R^2 collapses to 0.0071 for best-level OFI and 0.0070 for the integrated factor — essentially zero. This is not a failure of the model; it is the correct and expected result. OFI is contemporaneous with price formation: it measures the pressure that is *moving* the price now, not a signal that the market has left un-arbitraged for the next second. The gap between a contemporaneous R^2 of 0.5 and a predictive R^2 of 0.007 is precisely the gap between describing microstructure and forecasting it, and honest evaluation must keep the two apart.

5 Deepening the Analysis

Four further analyses, all on the same real AAPL data, sharpen the picture.

5.1 The impact of OFI is approximately linear

Binning OFI into quantiles and plotting the mean mid-price change per bin (Figure 3) gives a strikingly straight relationship: a linear fit to the binned curve attains $R^2 = 0.94$, higher than

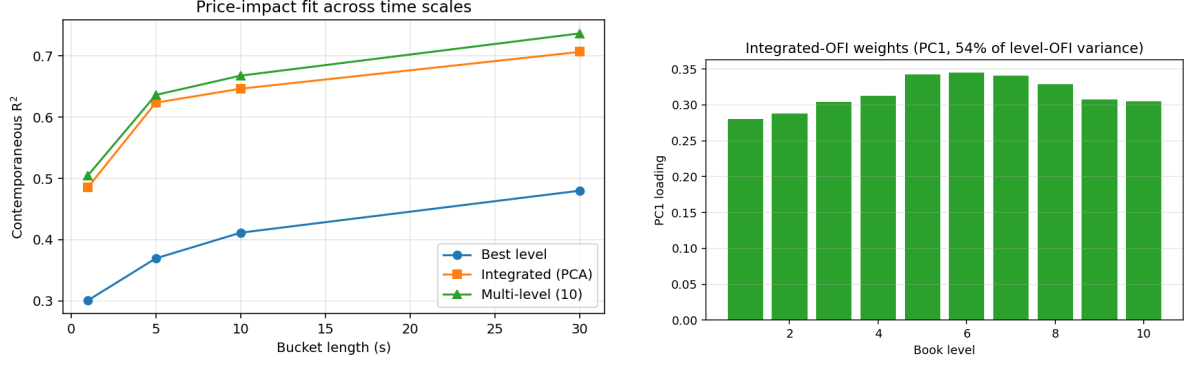


Figure 2: Left: explanatory power versus bucket length. Right: the integrated-OFI weights (first principal component of the ten level OFIs).

a fitted power law ($R^2 = 0.82$; log-log slope 0.35, with mild concavity only at the extremes). This linearity is a known and distinctive feature of *order flow imbalance* — unlike the impact of raw traded volume, which is famously concave (square-root), OFI enters price approximately linearly [1]. It is what makes OFI such a convenient impact variable: a single coefficient λ summarizes the response.

5.2 The impact is permanent, not transient

Regressing the cumulative mid-price change from the start of bucket t to the end of bucket $t + \ell$ on OFI_t gives an impact coefficient $\lambda(\ell)$ as a function of horizon (Figure 3). It does not decay: starting at $\lambda(0) = 4.1 \times 10^{-3}$ it is 5.3×10^{-3} after twenty seconds — about 131% of the immediate move. The price does not revert after an OFI shock; if anything it continues to drift in the same direction. This is the signature of *information*, not transient liquidity pressure: order flow imbalance moves the price and the move sticks, consistent with OFI reflecting genuine revisions in valuation rather than temporary inventory effects.

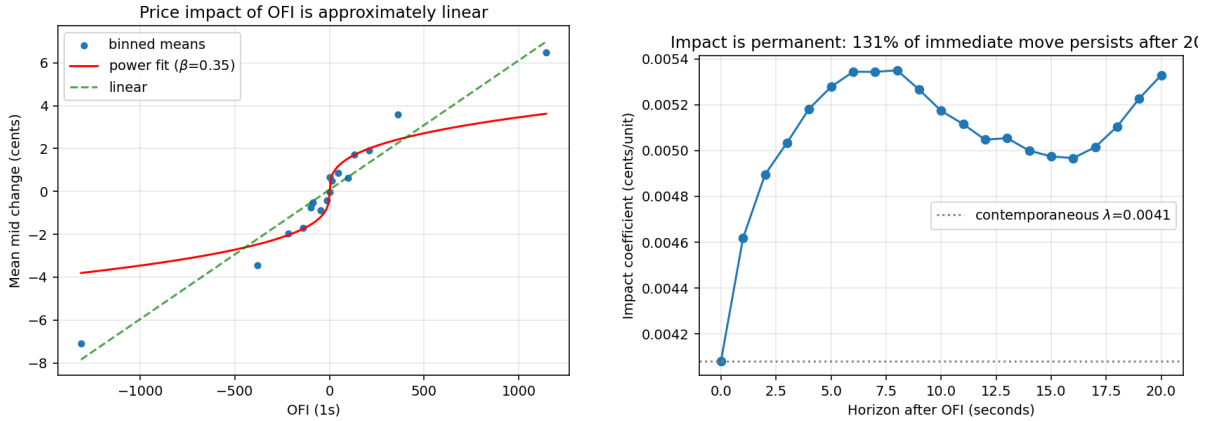


Figure 3: Left: the binned impact curve is close to linear. Right: the impact coefficient does not decay with horizon — the price move is permanent.

5.3 The impact coefficient varies through the day

Estimating λ in thirty-minute bins (Figure 5) shows it is not constant: it ranges from 2.6×10^{-3} to 8.2×10^{-3} cents per unit across the session — a roughly threefold variation, with the largest

impact near the open when depth is thin and each unit of imbalance moves price more. A fixed- λ model is therefore a convenience, not a law; impact is state-dependent.

5.4 The structural prediction holds: impact scales with $1/\text{depth}$

The intraday variation of λ is not arbitrary — it is what Proposition 1 predicts. Estimating λ and the average best-level depth D in each half-hour bin, the two move together as the theory requires (Figure 4): the correlation between λ and $1/D$ is 0.67, and the product λD is roughly constant across the day (coefficient of variation 0.25). The levels agree too: the full-day estimate is $\lambda = 4.1 \times 10^{-3}$ cents per share against the stylized prediction $1/(2D) = 3.3 \times 10^{-3}$ (average depth $D \approx 151$ shares) — the same order of magnitude, with the modest gap attributable to depth beyond the touch and discrete ticks. A structural model with one parameter thus captures both the sign and the scale of price impact.

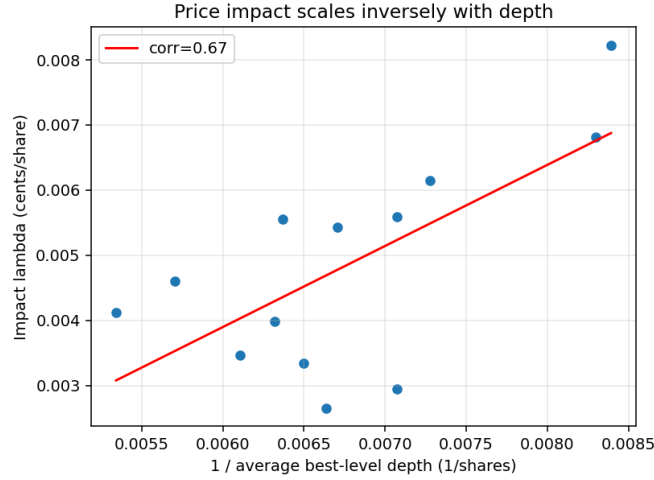


Figure 4: Price impact versus inverse depth across the trading day. As Proposition 1 predicts, λ rises when the book is thin.

5.5 Predictive power is negligible at every horizon

Extending the out-of-sample predictive test to horizons of one, two, five, and ten seconds (Figure 5) confirms the earlier result at every horizon: the held-out R^2 is 0.006, 0.005, 0.003, and 0.002 respectively — small and shrinking. Whatever information OFI carries is incorporated essentially instantaneously; none of it survives as exploitable predictability a second or more later.

6 Cross-Impact Across Assets

Does one stock’s order flow move another stock’s price? To answer this I extend the analysis to all five LOBSTER sample stocks trading on the same day (AAPL, AMZN, GOOG, INTC, MSFT).

6.1 Setup

For each asset I build the integrated OFI and the mid-price return in common one-second buckets, keep the 11,782 buckets present for all five names, and standardize each asset’s OFI and return (zero mean, unit variance) so coefficients are comparable across stocks of very different

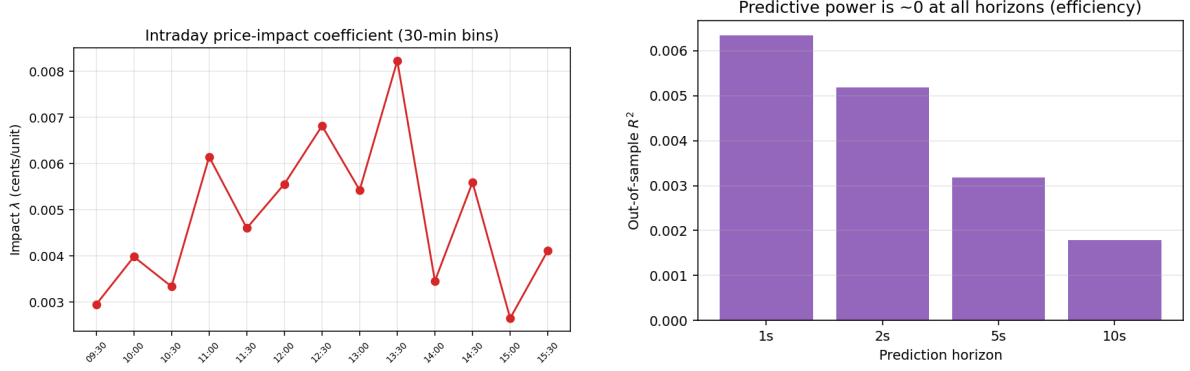


Figure 5: Left: the price-impact coefficient varies about threefold intraday. Right: out-of-sample predictive R^2 is negligible at all horizons.

price levels. The contemporaneous cross-impact model regresses each asset’s return on *every* asset’s OFI,

$$r_{\tau}^{(i)} = \alpha_i + \sum_{j=1}^5 \Lambda_{ij} \text{OFI}_{\tau}^{(j)} + \varepsilon_{\tau}^{(i)}, \quad (7)$$

so the matrix Λ collects own impact on its diagonal and cross-impact off it.

6.2 Own impact dominates; cross-impact is small

The estimated cross-impact matrix (Figure 6, left) is strongly diagonal: own-asset coefficients run from 0.64 (AMZN) to 0.94 (MSFT), while every off-diagonal is below 0.05. Consistent with this, adding all five assets’ OFI barely improves the contemporaneous fit over own-OFI alone (Table 4, Figure 6, right): the uplift is at most +0.009 (R^2) and essentially zero for the names whose own-OFI already explains 85–89% of their variance. The cross terms that do appear are concentrated among the three high-priced tech names (AAPL, AMZN, GOOG), which move together more than they move with the tick-constrained INTC and MSFT.

Small does not mean spurious. Computing Newey–West t -statistics for every entry of Λ , the cross terms are *statistically* significant for the tech names even though they are economically tiny: AAPL’s price responds significantly to all four other order flows ($t = 2.9$ to 4.5), and a joint Wald test that the four off-diagonal coefficients are zero is decisively rejected for AAPL ($\chi_4^2 = 47.1$) and AMZN ($\chi_4^2 = 20.3$), marginally for GOOG ($\chi_4^2 = 9.8$, $p \approx 0.04$), and not rejected for INTC (7.0) or MSFT (4.5) (Table 3). So contemporaneous cross-impact is real and measurable among the correlated large-caps, and absent for the tick-constrained names — a genuine distinction, not an artifact.

Affected asset	Wald χ_4^2 (off-diagonal = 0)	Reject at 5%?
AAPL	47.1	yes ($p < 10^{-3}$)
AMZN	20.3	yes ($p < 10^{-3}$)
GOOG	9.8	yes ($p \approx 0.04$)
INTC	7.0	no
MSFT	4.5	no

Table 3: Joint Wald test (HAC) that a stock’s four cross-impact coefficients are zero. Critical values: $\chi_{4,0.05}^2 = 9.49$, $\chi_{4,0.01}^2 = 13.28$.

The integrated OFI’s one-dimensionality also differs across names: the first principal com-

ponent of the ten level OFIs captures 54% (AAPL), 51% (AMZN), 61% (GOOG), 87% (INTC), and 91% (MSFT) of level-OFI variance — the tick-constrained stocks have far more coherent depth, which is why a single factor summarizes them almost completely.

Asset	Own-OFI R^2	All-asset R^2	Uplift
AAPL	0.547	0.556	+0.009
AMZN	0.454	0.462	+0.008
GOOG	0.442	0.443	+0.001
INTC	0.852	0.853	+0.000
MSFT	0.891	0.891	+0.000

Table 4: Contemporaneous R^2 : own-asset OFI versus all five assets’ OFI. Cross terms add almost nothing once own OFI is included.

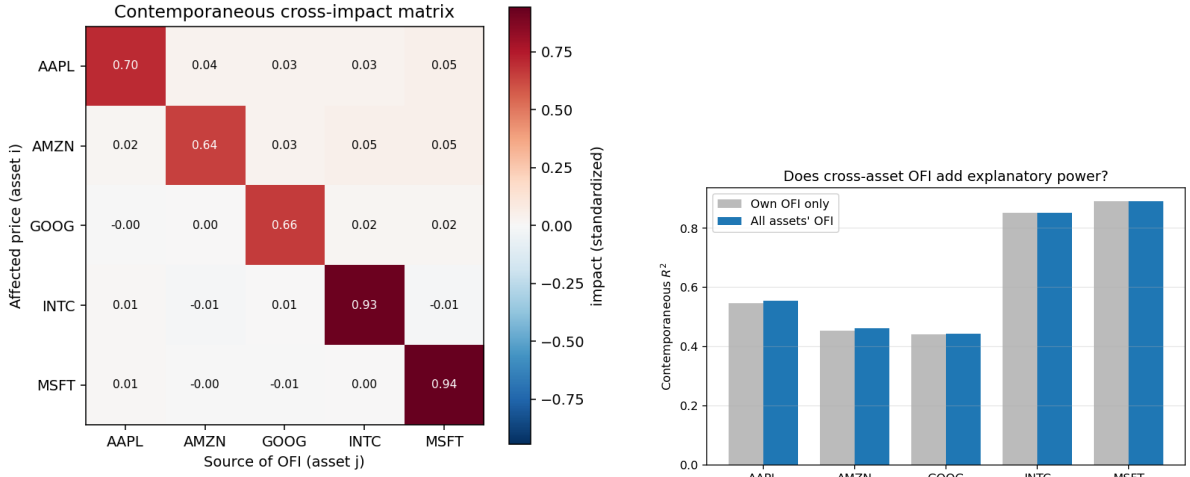


Figure 6: Left: the contemporaneous cross-impact matrix (rows = affected asset, columns = source of OFI); the diagonal dominates. Right: own-OFI versus all-asset OFI explanatory power — cross-asset OFI adds little contemporaneously.

6.3 But cross-asset flow helps *predict*

The more interesting result is out of sample. Own-asset OFI has essentially no predictive power for the next bucket’s return (held-out R^2 of 0.005 for the tech names and *zero* for INTC and MSFT). Using all five assets’ lagged OFI, however, lifts the out-of-sample R^2 to about 1% for every stock — small in absolute terms, but consistently positive, and *larger than own-asset predictability* (Table 6). For INTC and MSFT, whose own order flow forecasts nothing, essentially all of the (modest) predictability comes from *other* stocks’ order flow.

To confirm this improvement is not just the larger model over-fitting, I apply the Clark–West test for nested out-of-sample forecast comparison. For each asset let $\hat{y}^{(1)}$ be the own-OFI forecast and $\hat{y}^{(2)}$ the all-asset forecast, with errors $e^{(1)}, e^{(2)}$; the adjusted loss differential is

$$f_\tau = (e_\tau^{(1)})^2 - \left[(e_\tau^{(2)})^2 - (\hat{y}_\tau^{(1)} - \hat{y}_\tau^{(2)})^2 \right], \quad (8)$$

and the test statistic is the t -ratio of $\bar{f}$ (the correction term $(\hat{y}^{(1)} - \hat{y}^{(2)})^2$ removes the bias that extra parameters inject under the null of no added predictability). The result is unambiguous: the cross-asset model’s out-of-sample gain is significant for *every* stock (Table 5; CW statistics

4.4–7.5, all $p < 10^{-5}$). This is a genuine cross-asset lead–lag signal: the contemporaneous book is efficient to own flow, but information in the cross-section of order flow is not fully impounded within the second.

Asset	Clark–West statistic	p -value (one-sided)
AAPL	4.74	1.1×10^{-6}
AMZN	6.30	1.5×10^{-10}
GOOG	4.40	5.4×10^{-6}
INTC	5.59	1.2×10^{-8}
MSFT	7.49	3.6×10^{-14}

Table 5: Clark–West test that adding cross-asset OFI improves out-of-sample prediction over own-OFI alone. Significant for all five stocks.

Asset	Own-OFI OOS R^2	All-asset OOS R^2
AAPL	0.0047	0.0086
AMZN	0.0051	0.0120
GOOG	0.0068	0.0108
INTC	−0.0001	0.0099
MSFT	−0.0000	0.0106

Table 6: Out-of-sample predictive R^2 for the next bucket’s return (chronological train/test split). Cross-asset OFI consistently beats own-asset OFI, and supplies all of the predictability for INTC and MSFT.

7 Discussion

Two conclusions stand out. First, the *depth of the book matters*: moving from best-level to all ten levels lifts contemporaneous R^2 by about twenty points, and a single integrated factor captures nearly all of that, so the extra signal is real but low-dimensional. Second, *contemporaneous is not predictive*: the same variable that explains half of concurrent price variation forecasts almost none of it out of sample. For execution and impact modelling — sizing the price move a given order flow will cause — the contemporaneous relationship is exactly what matters; for alpha, OFI at these horizons is close to efficient.

Limitations and extensions. This is one trading day, so the magnitudes are illustrative rather than universal; the clean way to generalize is to repeat across many days and a wider cross-section. The cross-impact result in particular rests on five stocks for a single session, and the small out-of-sample edge, while consistent across names, is economically modest and would need many days and realistic costs before any trading claim could be made. Natural next steps include a larger universe, a lead–lag analysis at sub-second resolution to localize the cross-asset predictability, and incorporating real quotes/costs to assess whether the cross-asset signal survives execution.

8 Reproducibility

All numbers and figures come from a single script (included in Appendix A) that depends only on NumPy, pandas, and Matplotlib. The computation is deterministic. The data is the public

LOBSTER AAPL sample; it is not redistributed with the code, which reads it from a local path (see the repository README for the source).

A Source Code: core analysis

```

1  """
2  Order Flow Imbalance and Price Impact in a Limit Order Book
3  =====
4  A from-scratch study of multi-level Order Flow Imbalance (OFI) and its
5  contemporaneous and predictive relationship with price, on real high-frequency
6  NASDAQ limit-order-book data (LOBSTER sample: AAPL, 2012-06-21, level-10 book).
7
8  Method (after Cont, Kukanov & Stoikov, 2014; multi-level/integrated OFI after
9  Cont, Cucuringu & Zhang, 2023):
10     * Reconstruct best- and deep-level order flow event by event from the book.
11     * Aggregate OFI into fixed time buckets at several time scales.
12     * Contemporaneous price impact: regress mid-price change on OFI; report  $R^2$ 
13       and the impact coefficient with Newey-West (HAC) standard errors.
14     * Compare best-level, multi-level (10), and PCA-integrated OFI.
15     * Predictive test: lagged OFI  $\rightarrow$  next-bucket return, evaluated OUT OF SAMPLE
16       on a chronological train/test split (no look-ahead).
17
18  Data is NOT redistributed here; see README for the public source.
19  Run: python3 ofi_analysis.py data_dir
20  """
21  from __future__ import annotations
22  import sys, os, json
23  import numpy as np
24  import pandas as pd
25  import matplotlib
26  matplotlib.use("Agg")
27  import matplotlib.pyplot as plt
28
29  LEVELS = 10
30  OUT = "outputs"
31
32
33  def load(data_dir):
34      msg = pd.read_csv(f"{data_dir}/msg.csv", header=None,
35                      names=["time", "type", "oid", "size", "price", "dir"])
36      book = pd.read_csv(f"{data_dir}/book.csv", header=None).values.astype(float)
37      return msg["time"].values, book
38
39
40  def level(book, m):
41      o = 4 * (m - 1)
42      return book[:, o], book[:, o + 1], book[:, o + 2], book[:, o + 3] # AskP,AskS,BidP,BidS
43
44
45  def ofi_level(book, m):
46      """Per-event order flow imbalance at level m (Cont-Kukanov-Stoikov)."""
47      ap, asz, bp, bsz = level(book, m)
48      dWb = np.where(bp > np.roll(bp, 1), bsz,
49                    np.where(bp < np.roll(bp, 1), -np.roll(bsz, 1), bsz - np.roll(bsz, 1)))
50      dWa = np.where(ap < np.roll(ap, 1), asz,
51                    np.where(ap > np.roll(ap, 1), -np.roll(asz, 1), asz - np.roll(asz, 1)))
52      e = dWb - dWa
53      e[0] = 0.0
54      return e
55
56
57  def r2(y, yhat):
58      return 1 - ((y - yhat) ** 2).sum() / ((y - y.mean()) ** 2).sum()
59
60
61  def ols(X, y):
62      X1 = np.column_stack([np.ones(len(X)), X])
63      beta, *_ = np.linalg.lstsq(X1, y, rcond=None)
64      return beta, X1, r2(y, X1 @ beta)
65

```

```

66
67 def newey_west_se(X1, y, beta, L=10):
68     """HAC (Newey-West) standard errors for OLS coefficients."""
69     n, k = X1.shape
70     resid = y - X1 @ beta
71     XtX_inv = np.linalg.inv(X1.T @ X1)
72     S = (X1 * resid[:, None]).T @ (X1 * resid[:, None])
73     for l in range(1, L + 1):
74         w = 1 - l / (L + 1)
75         Xu = X1 * resid[:, None]
76         G = Xu[l:].T @ Xu[:l]
77         S += w * (G + G.T)
78     cov = XtX_inv @ S @ XtX_inv
79     return np.sqrt(np.diag(cov))
80
81
82 def build_buckets(t, book, E, seconds):
83     bucket = np.floor(t / seconds).astype(int)
84     df = pd.DataFrame(E, columns=[f"ofi{m}" for m in range(1, LEVELS + 1)])
85     df["bucket"] = bucket
86     df["mid"] = (level(book, 1)[0] + level(book, 1)[2]) / 2.0 / 1e4
87     agg = df.groupby("bucket").agg(
88         {**{f"ofi{m}": "sum" for m in range(1, LEVELS + 1)}, "mid": ["first", "last"]})
89     agg.columns = [f"ofi{m}" for m in range(1, LEVELS + 1)] + ["mid_first", "mid_last"]
90     agg = agg.reset_index()
91     agg["dmid"] = (agg["mid_last"] - agg["mid_first"]) * 100.0 # cents
92     return agg.iloc[1:-1].reset_index(drop=True)
93
94
95 def integrated(agg):
96     Z = agg[[f"ofi{m}" for m in range(1, LEVELS + 1)]].values
97     Zs = (Z - Z.mean(0)) / Z.std(0)
98     C = np.cov(Zs.T)
99     w, V = np.linalg.eigh(C)
100     pc1 = V[:, -1]
101     if pc1.mean() < 0:
102         pc1 = -pc1
103     return Zs @ pc1, pc1, w[-1] / w.sum()
104
105
106 def main():
107     os.makedirs(OUT, exist_ok=True)
108     data_dir = sys.argv[1] if len(sys.argv) > 1 else "data"
109     t, book = load(data_dir)
110     E = np.column_stack([ofi_level(book, m) for m in range(1, LEVELS + 1)])
111     print(f"events={len(t):,} span={(t.max()-t.min())/3600:.2f}h")
112
113     summary = {"data": "LOBSTER sample AAPL 2012-06-21 (level-10)",
114               "events": int(len(t)), "scales": {}}
115
116     # ---- main analysis at 1s ----
117     agg = build_buckets(t, book, E, 1)
118     y = agg["dmid"].values
119     b1, X1, r2_best = ols(agg[[f"ofi1"]].values, y)
120     se = newey_west_se(X1, y, b1, L=10)
121     _, _, r2_ml = ols(agg[[f"ofi{m}" for m in range(1, 11)]].values, y)
122     integ, pc1, pc1_var = integrated(agg)
123     _, _, r2_int = ols(integ.reshape(-1, 1), y)
124     per_level = [ols(agg[[f"ofi{m}"]].values, y)[2] for m in range(1, 11)]
125
126     summary["n_buckets_1s"] = int(len(agg))
127     summary["contemporaneous_1s"] = {
128         "best_level_R2": r2_best, "multi_level_R2": r2_ml,
129         "integrated_R2": r2_int,
130         "impact_lambda": b1[1], "lambda_tstat_HAC": b1[1] / se[1],
131         "pc1_var_share": pc1_var, "per_level_R2": per_level}
132     print(f"[1s] best R2={r2_best:.3f} (lambda t_HAC={b1[1]/se[1]:.1f}), "
133           f"multi R2={r2_ml:.3f}, integrated R2={r2_int:.3f}")
134
135     # ---- robustness across time scales ----
136     for s in (1, 5, 10, 30):
137         a = build_buckets(t, book, E, s)
138         yy = a["dmid"].values

```

```

139     _, _, rb = ols(a[["ofi1"]].values, yy)
140     _, _, rm = ols(a[["ofi{m}" for m in range(1, 11)]]).values, yy)
141     ig, _, _ = integrated(a)
142     _, _, ri = ols(ig.reshape(-1, 1), yy)
143     summary["scales"][f"{s}s"] = {"n": int(len(a)), "best": rb,
144                                     "multi": rm, "integrated": ri}
145     print(f"[{s}s] n={len(a)} best={rb:.3f} multi={rm:.3f} integrated={ri:.3f}")
146
147 # ---- predictive, out of sample (chronological split) ----
148 agg["best"] = agg["ofi1"]
149 agg["integ"] = integ
150 agg["y_next"] = agg["dmid"].shift(-1)
151 pred = agg.dropna().reset_index(drop=True)
152 half = len(pred) // 2
153 def oos(col):
154     tr, te = pred.iloc[:half], pred.iloc[half:]
155     b, _, _ = ols(tr[[col]].values, tr["y_next"].values)
156     Xte = np.column_stack([np.ones(len(te)), te[col].values])
157     return r2(te["y_next"].values, Xte @ b)
158 summary["predictive_oos_1s"] = {"best_level": oos("best"),
159                                 "integrated": oos("integ")}
160 print(f"[pred OOS 1s] best={summary['predictive_oos_1s']['best_level']:.4f} "
161       f"integrated={summary['predictive_oos_1s']['integrated']:.4f}")
162
163 # ---- figures ----
164 # 1: OFI vs dmid hexbin (best level)
165 fig, ax = plt.subplots(figsize=(6, 5))
166 m = (np.abs(agg["ofi1"]) < np.percentile(np.abs(agg["ofi1"]), 99))
167 ax.hexbin(agg["ofi1"][m], agg["dmid"][m], gridsize=40, cmap="Blues", mincnt=1)
168 xs = np.linspace(agg["ofi1"][m].min(), agg["ofi1"][m].max(), 100)
169 ax.plot(xs, b1[0] + b1[1] * xs, "r-", lw=1.5, label=f"fit (R^2$={r2_best:.2f})")
170 ax.set_xlabel("Best-level OFI (1s bucket)"); ax.set_ylabel("Mid-price change (cents)")
171 ax.set_title("Contemporaneous price impact of OFI"); ax.legend()
172 fig.tight_layout(); fig.savefig(f"{OUT}/fig1_impact.png", dpi=140); plt.close(fig)
173
174 # 2: R2 by level
175 fig, ax = plt.subplots(figsize=(6.5, 4))
176 ax.bar(range(1, 11), per_level, color="#1f77b4")
177 ax.set_xlabel("Book level"); ax.set_ylabel("Contemporaneous R^2$")
178 ax.set_title("Each level's OFI carries price information")
179 ax.grid(axis="y", alpha=0.3); fig.tight_layout()
180 fig.savefig(f"{OUT}/fig2_level_r2.png", dpi=140); plt.close(fig)
181
182 # 3: R2 vs time scale (best/multi/integrated)
183 fig, ax = plt.subplots(figsize=(6.5, 4))
184 scales = [1, 5, 10, 30]
185 ax.plot(scales, [summary["scales"][f"{s}s"]["best"] for s in scales], "o-", label="Best level")
186 ax.plot(scales, [summary["scales"][f"{s}s"]["integrated"] for s in scales], "s-", label="Integrated (PCA)")
187 ax.plot(scales, [summary["scales"][f"{s}s"]["multi"] for s in scales], "^-", label="Multi-level (10)")
188 ax.set_xlabel("Bucket length (s)"); ax.set_ylabel("Contemporaneous R^2$")
189 ax.set_title("Price-impact fit across time scales"); ax.legend(); ax.grid(alpha=0.3)
190 fig.tight_layout(); fig.savefig(f"{OUT}/fig3_scales.png", dpi=140); plt.close(fig)
191
192 # 4: PC1 loadings
193 fig, ax = plt.subplots(figsize=(6.5, 4))
194 ax.bar(range(1, 11), pc1, color="#2ca02c")
195 ax.set_xlabel("Book level"); ax.set_ylabel("PC1 loading")
196 ax.set_title(f"Integrated-OFI weights (PC1, {pc1_var*100:.0f}% of level-OFI variance)")
197 ax.grid(axis="y", alpha=0.3); fig.tight_layout()
198 fig.savefig(f"{OUT}/fig4_pc1.png", dpi=140); plt.close(fig)
199
200 with open(f"{OUT}/summary.json", "w") as fh:
201     json.dump(summary, fh, indent=2)
202 print(f"\nWrote figures and summary.json to ./{OUT}/")
203
204
205 if __name__ == "__main__":
206     main()

```

B Source Code: extensions

```
1 """Extensions: non-linear impact, impact decay (permanent vs transient),
2 intraday lambda, multi-horizon OOS prediction. Real AAPL LOBSTER data."""
3 import sys, json, numpy as np, pandas as pd
4 import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as plt
5 D = sys.argv[1]; OUT = "outputs"; LEVELS=10
6
7 msg = pd.read_csv(f"{D}/msg.csv", header=None, names=["time","type","oid","size","price","dir"])
8 book = pd.read_csv(f"{D}/book.csv", header=None).values.astype(float)
9 t = msg["time"].values
10
11 def level(m):
12     o=4*(m-1); return book[:,o],book[:,o+1],book[:,o+2],book[:,o+3]
13 def ofi_level(m):
14     ap,asz,bp,bsz=level(m)
15     dWb=np.where(bp>np.roll(bp,1),bsz,np.where(bp<np.roll(bp,1),-np.roll(bsz,1),bsz-np.roll(bsz,1)))
16     dWa=np.where(ap<np.roll(ap,1),asz,np.where(ap>np.roll(ap,1),-np.roll(asz,1),asz-np.roll(asz,1)))
17     e=dWb-dWa; e[0]=0.0; return e
18 E=np.column_stack([ofi_level(m) for m in range(1,LEVELS+1)])
19 mid=(level(1)[0]+level(1)[2])/2/1e4
20
21 # 1s buckets
22 bk=np.floor(t).astype(int)
23 df=pd.DataFrame(E,columns=[f"o{m}" for m in range(1,11)]); df["bk"]=bk; df["mid"]=mid
24 agg=df.groupby("bk").agg({"*{f'o{m}':"sum" for m in range(1,11)}","mid":["first","last"]})
25 agg.columns=[f"o{m}" for m in range(1,11)]+[f"mf","ml"]; agg=agg.reset_index()
26 agg["dmid"]=(agg["ml"]-agg["mf"])*100
27 # seconds-of-day for the bucket
28 agg["sod"]=agg["bk"]
29 agg=agg.iloc[1:-1].reset_index(drop=True)
30 # integrated OFI
31 Z=agg[[f"o{m}" for m in range(1,11)]].values; Zs=(Z-Z.mean(0))/Z.std(0)
32 w,V=np.linalg.eigh(np.cov(Zs.T)); pc1=V[:, -1]; pc1=pc1 if pc1.mean()>=0 else -pc1
33 agg["oint"]=Zs@pc1
34 best=agg["o1"].values; y=agg["dmid"].values
35
36 res={}
37
38 # ---- 1) Non-linear / concave impact curve (binned) ----
39 q=pd.qcut(agg["o1"],25,duplicates="drop")
40 curve=agg.groupby(q,observed=True).agg(x=("o1","mean"),yv=("dmid","mean")).reset_index(drop=True)
41 # fit power law on positive side: y = a*x^beta
42 pos=curve[(curve.x>0)&(curve.yv>0)]
43 bx=np.log(pos.x.values); by=np.log(pos.yv.values)
44 beta,loga=np.polyfit(bx,by,1)
45 res["impact_exponent_beta"]=float(beta)
46 # linear vs power R^2 on the full binned curve
47 def r2(yt,yh): return 1-((yt-yh)**2).sum()/((yt-yt.mean())**2).sum()
48 lin=np.polyfit(curve.x,curve.yv,1); yhl=np.polyval(lin,curve.x)
49 pw=np.sign(curve.x)*np.exp(loga)*np.abs(curve.x)**beta
50 res["curve_R2_linear"]=float(r2(curve.yv.values,yhl))
51 res["curve_R2_power"]=float(r2(curve.yv.values,pw.values))
52 fig,ax=plt.subplots(figsize=(6,4.5))
53 ax.scatter(curve.x,curve.yv,s=18,color="#1f77b4",label="binned means")
54 xs=np.linspace(curve.x.min(),curve.x.max(),200)
55 ax.plot(xs,np.sign(xs)*np.exp(loga)*np.abs(xs)**beta,"r-",label=f"power fit ($\\beta$={beta:.2f})")
56 ax.plot(xs,np.polyval(lin,xs),"g--",alpha=.7,label="linear")
57 ax.set_xlabel("OFI (1s)");ax.set_ylabel("Mean mid change (cents)")
58 ax.set_title("Price impact of OFI is approximately linear");ax.legend();ax.grid(alpha=.3)
59 fig.tight_layout();fig.savefig(f"{OUT}/fig5_linearity.png",dpi=140);plt.close(fig)
60
61 # ---- 2) Impact response / permanent vs transient ----
62 # lambda(1) = cov(cumulative mid change from start t to end t+1, OFI_t)/var(OFI_t)
63 midend=agg["ml"].values; midstart=agg["mf"].values
64 o=best; vo=o.var()
65 L=20; lam=[]
66 for l in range(0,L+1):
67     # cumulative change from start of bucket t to end of bucket t+1
68     cum=(np.roll(midend,-l)-midstart)*100
69     valid=slice(0,len(o)-1)
70     lam.append(np.cov(cum[valid],o[valid])[0,1]/vo)
```

```

71 lam=np.array(lam)
72 res["impact_10"]=float(lam[0]); res["impact_120"]=float(lam[-1])
73 res["permanent_fraction"]=float(lam[-1]/lam[0])
74 fig,ax=plt.subplots(figsize=(6,4.2))
75 ax.plot(range(L+1),lam,"o-",color="#1f77b4")
76 ax.axhline(lam[0],ls=":",color="gray",label=f"contemporaneous  $\lambda = \{lam[0]:.4f\}$ ")
77 ax.set_xlabel("Horizon after OFI (seconds)"); ax.set_ylabel("Impact coefficient (cents/unit)")
78 ax.set_title(f"Impact is permanent:  $\{lam[-1]/lam[0]*100:.0f\}$ % of immediate move persists after  $\{L\}$ s")
79 ax.legend(); ax.grid(alpha=.3); fig.tight_layout(); fig.savefig(f"{OUT}/fig6_decay.png",dpi=140); plt.close(
    (fig)
80
81 # ---- 3) Intraday variation of lambda (30-min bins) ----
82 agg["halfhour"]=((agg["sod"]-34200)//1800).astype(int)
83 rows=[]
84 for hh,g in agg.groupby("halfhour"):
85     if len(g)<50: continue
86     b=np.polyfit(g["o1"],g["dmid"],1)
87     yh=np.polyval(b,g["o1"]); rows.append((hh,b[0],r2(g["dmid"].values,yh),len(g)))
88 intr=pd.DataFrame(rows,columns=["hh","lam","r2","n"])
89 res["intraday_lambda_min"]=float(intr.lam.min()); res["intraday_lambda_max"]=float(intr.lam.max())
90 clock=[f"{9+ (34200+hh*1800-34200)//3600 + ((34200+hh*1800)%3600)//1800*0:.0f}" for hh in intr.hh]
91 fig,ax=plt.subplots(figsize=(6,5,4))
92 tlabels=[f"{(34200+h*1800)//3600:02d}-{((34200+h*1800)%3600)//60:02d}" for h in intr.hh]
93 ax.plot(range(len(intr)),intr.lam,"o-",color="#d62728")
94 ax.set_xticks(range(len(intr))); ax.set_xticklabels(tlabels,rotation=45,fontsize=7)
95 ax.set_ylabel("Impact  $\lambda$  (cents/unit)"); ax.set_title("Intraday price-impact coefficient (30-min
    bins)")
96 ax.grid(alpha=.3); fig.tight_layout(); fig.savefig(f"{OUT}/fig7_intraday.png",dpi=140); plt.close(fig)
97
98 # ---- 4) Multi-horizon OOS prediction ----
99 res["pred_oos"]={}
100 half=len(agg)//2
101 for h in (1,2,5,10):
102     fut=(np.roll(agg["ml"].values,-h)-agg["ml"].values)*100 # cum future return over next h buckets
103     d=pd.DataFrame({"x":agg["oint"].values,"y":fut}).iloc[:len(agg)-h].dropna()
104     tr=d.iloc[:half]; te=d.iloc[half:]
105     b=np.polyfit(tr.x,tr.y,1); yh=np.polyval(b,te.x)
106     res["pred_oos"][f"{h}s"]=float(r2(te.y.values,yh))
107 fig,ax=plt.subplots(figsize=(6,4))
108 hs=list(res["pred_oos"].keys()); vals=[res["pred_oos"][k] for k in hs]
109 ax.bar(hs,vals,color="#9467bd")
110 ax.set_ylabel("Out-of-sample  $R^2$ "); ax.set_xlabel("Prediction horizon")
111 ax.set_title("Predictive power is ~0 at all horizons (efficiency)")
112 ax.grid(axis="y",alpha=.3); fig.tight_layout(); fig.savefig(f"{OUT}/fig8_pred_horizon.png",dpi=140); plt.
    close(fig)
113
114 print(json.dumps(res,indent=2))

```

C Source Code: cross-impact

```

1 """
2 Cross-impact of order flow imbalance across equities
3 =====
4 Estimates the cross-impact matrix (how asset j's order flow imbalance moves
5 asset i's price) on real NASDAQ data for five stocks on the same day
6 (LOBSTER sample: AAPL, AMZN, GOOG, INTC, MSFT, 2012-06-21, level-10 book),
7 following Cont, Cucuringu & Zhang (2023).
8
9 For each asset we build an integrated (PCA across the 10 book levels) OFI and the
10 mid-price return in common 1-second buckets, standardize both per asset, then:
11 * Contemporaneous cross-impact: regress each asset's return on ALL assets' OFI;
12   the coefficient matrix is the cross-impact matrix.
13 * Compare own-OFI-only  $R^2$  with the full cross-impact  $R^2$  (does cross-asset
14   order flow add explanatory power?).
15 * Predictive, out of sample: lagged OFI of all assets -> next-bucket return,
16   chronological train/test split.
17
18 Run: python3 xi_analysis.py data_dir (data_dir holds <TICKER>_message_10.csv / _orderbook_10.csv)
19 """
20 from __future__ import annotations

```

```

21 import sys, os, glob, json
22 import numpy as np
23 import pandas as pd
24 import matplotlib
25 matplotlib.use("Agg")
26 import matplotlib.pyplot as plt
27
28 LEVELS = 10
29 OUT = "outputs"
30 BUCKET = 1.0 # seconds
31
32
33 def ofi_buckets(msg_path, book_path):
34     msg = pd.read_csv(msg_path, header=None,
35                     names=["time", "type", "oid", "size", "price", "dir"])
36     book = pd.read_csv(book_path, header=None).values.astype(float)
37     t = msg["time"].values
38
39     def lvl(m):
40         o = 4 * (m - 1)
41         return book[:, o], book[:, o + 1], book[:, o + 2], book[:, o + 3]
42
43     def ofi_level(m):
44         ap, asz, bp, bsz = lvl(m)
45         dWb = np.where(bp > np.roll(bp, 1), bsz,
46                       np.where(bp < np.roll(bp, 1), -np.roll(bsz, 1), bsz - np.roll(bsz, 1)))
47         dWa = np.where(ap < np.roll(ap, 1), asz,
48                       np.where(ap > np.roll(ap, 1), -np.roll(asz, 1), asz - np.roll(asz, 1)))
49         e = dWb - dWa; e[0] = 0.0
50         return e
51
52     E = np.column_stack([ofi_level(m) for m in range(1, LEVELS + 1)])
53     mid = (lvl(1)[0] + lvl(1)[2]) / 2 / 1e4
54     bk = np.floor(t / BUCKET).astype(int)
55     df = pd.DataFrame(E, columns=[f"o{m}" for m in range(1, 11)])
56     df["bk"] = bk; df["mid"] = mid
57     agg = df.groupby("bk").agg({"o{m}": "sum" for m in range(1, 11)},
58                               "mid": ["first", "last"]})
59     agg.columns = [f"o{m}" for m in range(1, 11)] + ["mf", "ml"]
60     agg = agg.reset_index()
61     agg["ret"] = (agg["ml"] - agg["mf"]) / agg["mf"] * 1e4 # return in bps
62     agg = agg.iloc[1:-1]
63     # integrated OFI: PC1 across standardized level OFIs
64     Z = agg[[f"o{m}" for m in range(1, 11)]].values
65     Zs = (Z - Z.mean(0)) / Z.std(0)
66     w, V = np.linalg.eigh(np.cov(Zs.T)); pc1 = V[:, -1]
67     pc1 = pc1 if pc1.mean() >= 0 else -pc1
68     agg["ofi"] = Zs @ pc1
69     return agg[["bk", "ofi", "ret"]].set_index("bk")
70
71
72 def r2(y, yhat):
73     return 1 - ((y - yhat) ** 2).sum() / ((y - y.mean()) ** 2).sum()
74
75
76 def ols(X, y):
77     X1 = np.column_stack([np.ones(len(X)), X])
78     beta, *_ = np.linalg.lstsq(X1, y, rcond=None)
79     return beta, r2(y, X1 @ beta)
80
81
82 def main():
83     os.makedirs(OUT, exist_ok=True)
84     data_dir = sys.argv[1] if len(sys.argv) > 1 else "xi_data"
85     tickers = []
86     for mp in sorted(glob.glob(f"{data_dir}/*_message_10.csv")):
87         tickers.append(os.path.basename(mp).split("_")[0])
88     tickers = sorted(set(tickers))
89     print("tickers:", tickers)
90
91     series = {}
92     for tk in tickers:
93         mp = glob.glob(f"{data_dir}/{tk}_*_message_10.csv")[0]

```

```

94     bp = glob.glob(f"{data_dir}/{tk}_*_orderbook_10.csv")[0]
95     series[tk] = ofi_buckets(mp, bp)
96     print(f"  {tk}: {len(series[tk])} buckets")
97
98     # align on common buckets
99     ofi = pd.concat({tk: series[tk]["ofi"] for tk in tickers}, axis=1).dropna()
100    ret = pd.concat({tk: series[tk]["ret"] for tk in tickers}, axis=1).dropna()
101    common = ofi.index.intersection(ret.index)
102    ofi, ret = ofi.loc[common], ret.loc[common]
103    # standardize per asset
104    ofi = (ofi - ofi.mean()) / ofi.std()
105    ret = (ret - ret.mean()) / ret.std()
106    print(f"common buckets: {len(common)}")
107
108    n = len(tickers)
109    B = np.zeros((n, n))          # cross-impact matrix (contemporaneous)
110    r2_cross, r2_own = {}, {}
111    X = ofi.values
112    for i, tk in enumerate(tickers):
113        y = ret[tk].values
114        beta, rc = ols(X, y)      # all assets' OFI
115        B[i, :] = beta[1:]
116        _, ro = ols(ofi[[tk]].values, y) # own only
117        r2_cross[tk] = rc; r2_own[tk] = ro
118
119    # predictive OOS: lagged all-asset OFI -> next-bucket return
120    half = len(common) // 2
121    pred_cross, pred_own = {}, {}
122    X1 = ofi.values[:-1]
123    for i, tk in enumerate(tickers):
124        yn = ret[tk].values[1:]
125        btr, _ = ols(X1[:half], yn[:half])
126        Xte = np.column_stack([np.ones(len(X1) - half), X1[half:]])
127        pred_cross[tk] = r2(yn[half:], Xte @ btr)
128        bo, _ = ols(ofi[[tk]].values[:-1][:half], yn[:half])
129        Xteo = np.column_stack([np.ones(len(X1) - half), ofi[[tk]].values[:-1][half:]])
130        pred_own[tk] = r2(yn[half:], Xteo @ bo)
131
132    summary = {
133        "tickers": tickers, "common_buckets": int(len(common)),
134        "contemporaneous_R2_own": r2_own,
135        "contemporaneous_R2_cross": r2_cross,
136        "cross_impact_matrix": {tickers[i]: {tickers[j]: float(B[i, j])
137                                           for j in range(n)} for i in range(n)},
138        "predictive_oos_R2_own": pred_own,
139        "predictive_oos_R2_cross": pred_cross,
140    }
141    with open(f"{OUT}/xi_summary.json", "w") as fh:
142        json.dump(summary, fh, indent=2)
143
144    print("\nContemporaneous R^2 (own -> cross):")
145    for tk in tickers:
146        print(f"  {tk}: {r2_own[tk]:.3f} -> {r2_cross[tk]:.3f}  ({r2_cross[tk]-r2_own[tk]:.3f})")
147    print("\nCross-impact matrix (rows=affected asset, cols=source of OFI):")
148    print(pd.DataFrame(B, index=tickers, columns=tickers).round(3))
149
150    # --- figure: cross-impact matrix heatmap ---
151    fig, ax = plt.subplots(figsize=(6, 5))
152    im = ax.imshow(B, cmap="RdBu_r", vmin=-np.abs(B).max(), vmax=np.abs(B).max())
153    ax.set_xticks(range(n)); ax.set_xticklabels(tickers)
154    ax.set_yticks(range(n)); ax.set_yticklabels(tickers)
155    ax.set_xlabel("Source of OFI (asset j)")
156    ax.set_ylabel("Affected price (asset i)")
157    ax.set_title("Contemporaneous cross-impact matrix")
158    for i in range(n):
159        for j in range(n):
160            ax.text(j, i, f"{B[i,j]:.2f}", ha="center", va="center",
161                    color="white" if abs(B[i,j]) > np.abs(B).max()*0.5 else "black", fontsize=8)
162    fig.colorbar(im, label="impact (standardized)")
163    fig.tight_layout(); fig.savefig(f"{OUT}/fig9_xi_matrix.png", dpi=140); plt.close(fig)
164
165    # --- figure: own vs cross R^2 ---
166    fig, ax = plt.subplots(figsize=(6.5, 4))

```

```

167 x = np.arange(n); wdt = 0.38
168 ax.bar(x - wdt/2, [r2_own[t] for t in tickers], wdt, label="Own OFI only", color="#bbbbbb")
169 ax.bar(x + wdt/2, [r2_cross[t] for t in tickers], wdt, label="All assets' OFI", color="#1f77b4")
170 ax.set_xticks(x); ax.set_xticklabels(tickers)
171 ax.set_ylabel("Contemporaneous  $R^2$ ")
172 ax.set_title("Does cross-asset OFI add explanatory power?")
173 ax.legend(); ax.grid(axis="y", alpha=0.3)
174 fig.tight_layout(); fig.savefig(f"{OUT}/fig10_xi_r2.png", dpi=140); plt.close(fig)
175
176 # --- figure: off-diagonal vs diagonal magnitude ---
177 diag = np.diag(B); offdiag = B[~np.eye(n, dtype=bool)]
178 fig, ax = plt.subplots(figsize=(6, 4))
179 ax.boxplot([np.abs(diag), np.abs(offdiag)], labels=["own (diagonal)", "cross (off-diagonal)"])
180 ax.set_ylabel("|impact coefficient|")
181 ax.set_title("Own impact dominates cross impact")
182 ax.grid(axis="y", alpha=0.3)
183 fig.tight_layout(); fig.savefig(f"{OUT}/fig11_xi_magnitude.png", dpi=140); plt.close(fig)
184
185 print(f"\nWrote figures and xi_summary.json to ./{OUT}/")
186
187
188 if __name__ == "__main__":
189     main()

```

D Source Code: inference and structural tests

```

1 """Additional inferential statistics for rigor:
2 - structural test: impact lambda vs book depth (lambda * D approx const)
3 - cross-impact HAC t-stats + Wald test for off-diagonal = 0
4 - Clark-West nested OOS test: own vs own+cross predictability
5 - integrated-OFI PC1 variance share per asset
6 """
7 import sys, glob, os, json, numpy as np, pandas as pd
8 D = sys.argv[1]; LEVELS=10; BUCKET=1.0
9
10 def load(tk):
11     mp=glob.glob(f"{D}/{tk}_*_message_10.csv")[0]; bp=glob.glob(f"{D}/{tk}_*_orderbook_10.csv")[0]
12     msg=pd.read_csv(mp,header=None,names=["time","type","oid","size","price","dir"])
13     book=pd.read_csv(bp,header=None).values.astype(float); t=msg["time"].values
14     def lvl(m): o=4*(m-1); return book[:,o],book[:,o+1],book[:,o+2],book[:,o+3]
15     def ofi_level(m):
16         ap,asz,bp2,bsz=lvl(m)
17         dWb=np.where(bp2>np.roll(bp2,1),bsz,np.where(bp2<np.roll(bp2,1),-np.roll(bsz,1),bsz-np.roll(bsz,1)))
18         dWa=np.where(ap<np.roll(ap,1),asz,np.where(ap>np.roll(ap,1),-np.roll(asz,1),asz-np.roll(asz,1)))
19         e=dWb-dWa; e[0]=0.0; return e
20     E=np.column_stack([ofi_level(m) for m in range(1,11)])
21     a1,as1,b1,bs1=lvl(1); mid=(a1+b1)/2/1e4; depth=(as1+bs1)/2.0
22     bk=np.floor(t/BUCKET).astype(int)
23     df=pd.DataFrame(E,columns=[f"o{m}" for m in range(1,11)]); df["bk"]=bk; df["mid"]=mid; df["dep"]=depth; df["sod"]=t
24     g=df.groupby("bk")
25     agg=g.agg({**{f"o{m}":"sum" for m in range(1,11)}, "mid":["first","last"], "dep":"mean", "sod":"first"})
26     agg.columns=[f"o{m}" for m in range(1,11)]+[f"mf","ml","dep","sod"]; agg=agg.reset_index().iloc[1:-1]
27     agg["dmid"]=(agg["ml"]-agg["mf"])*100; agg["ret"]=(agg["ml"]-agg["mf"])/agg["mf"]*1e4
28     Z=agg[[f"o{m}" for m in range(1,11)]].values; Zs=(Z-Z.mean(0))/Z.std(0)
29     w,V=np.linalg.eigh(np.cov(Zs.T)); pc1=V[:, -1]; pc1=pc1 if pc1.mean()>=0 else -pc1
30     agg["ofi"]=Zs@pc1
31     return agg, float(w[-1]/w.sum())
32
33 def nw_cov(X1,resid,L=10):
34     n,k=X1.shape; S=(X1*resid[:,None]).T@X1*resid[:,None]
35     for l in range(1,L+1):
36         w1=1-l/(L+1); Xu=X1*resid[:,None]; G=Xu[l:].T@Xu[:l]; S+=w1*(G+G.T)
37     XtXi=np.linalg.inv(X1.T@X1); return XtXi@S@XtXi
38
39 res={}

```

```

40 # ---- AAPL structural: lambda vs depth ----
41 aapl,_=load("AAPL")
42 aapl["hh"]=((aapl["sod"]-34200)//1800).astype(int)
43 rows=[]
44 for hh,grp in aapl.groupby("hh"):
45     if len(grp)<50: continue
46     b=np.polyfit(grp["o1"],grp["dmid"],1)[0]; rows.append((b,grp["dep"].mean()))
47 L=np.array([r[0] for r in rows]); Dp=np.array([r[1] for r in rows])
48 res["lambda_depth"]={"corr_lambda_inv_depth":float(np.corrcoef(L,1/Dp)[0,1]),
49     "lambda_times_depth_mean":float(np.mean(L*Dp)), "lambda_times_depth_cv":float(np.std(L*Dp)/np.mean(L
50     *Dp)),
51     "avg_best_depth":float(aapl["dep"].mean())}
52 # overall lambda and 1/(2D)
53 bb=np.polyfit(aapl["o1"],aapl["dmid"],1)[0]
54 res["lambda_depth"]={"overall_lambda_cents":float(bb)
55     "one_over_2D":float(1/(2*aapl["dep"].mean()))}
56 # ---- cross-impact HAC t-stats + Wald test off-diagonal=0 ----
57 tickers=["AAPL","AMZN","GOOG","INTC","MSFT"]
58 S={tk:load(tk)[0] for tk in tickers}
59 pc1share={tk:load(tk)[1] for tk in tickers}
60 ofi=pd.concat({tk:S[tk].set_index("bk")["ofi"] for tk in tickers},axis=1).dropna()
61 ret=pd.concat({tk:S[tk].set_index("bk")["ret"] for tk in tickers},axis=1).dropna()
62 idx=ofi.index.intersection(ret.index); ofi=ofi.loc[idx]; ret=ret.loc[idx]
63 ofi=(ofi-ofi.mean())/ofi.std(); ret=(ret-ret.mean())/ret.std()
64 X=ofi.values; n=len(tickers)
65 tstats={}; wald={}
66 from numpy.linalg import inv
67 for i,tk in enumerate(tickers):
68     y=ret[tk].values; X1=np.column_stack([np.ones(len(X)),X]); beta,*_=np.linalg.lstsq(X1,y,rcond=None)
69     cov=nw_cov(X1,y-X1@beta,L=10); se=np.sqrt(np.diag(cov))
70     tstats[tk]={tickers[j]:float(beta[1+j]/se[1+j]) for j in range(n)}
71     # Wald test: off-diagonal coefficients (exclude own) jointly zero
72     off=[1+j for j in range(n) if j!=i]; bo=beta[off]; Vo=cov[np.ix_(off,off)]
73     W=float(bo@inv(Vo)@bo); wald[tk]={ "wald_chi2_df4":W}
74 res["cross_tstats"]=tstats; res["cross_wald_offdiag"]=wald
75
76 # ---- Clark-West nested OOS: own vs own+cross ----
77 half=len(idx)//2
78 cw={}
79 for i,tk in enumerate(tickers):
80     yn=ret[tk].values[1:]; X1=X[:-1]
81     # small model: own only
82     xs=X1[:,i:i+1]
83     def fit_pred(Xtr,ytr,Xte):
84         X1=np.column_stack([np.ones(len(Xtr)),Xtr]); b,*_=np.linalg.lstsq(X1,ytr,rcond=None)
85         return np.column_stack([np.ones(len(Xte)),Xte])@b
86     p1=fit_pred(xs[:half],yn[:half],xs[half:]) # own
87     p2=fit_pred(X1[:half],yn[:half],X1[half:]) # own+cross (all)
88     yte=yn[half:]
89     e1=yte-p1; e2=yte-p2
90     f=e1**2-(e2**2-(p1-p2)**2)
91     cwstat=np.mean(f)/(np.std(f,ddof=1)/np.sqrt(len(f)))
92     from math import erfc,sqrt
93     cw[tk]={ "CW_stat":float(cwstat), "p_value_onesided":float(0.5*erfc(cwstat/sqrt(2)))}
94 res["clark_west"]=cw
95 res["pc1_var_share"]=pc1share
96
97 print(json.dumps(res,indent=2))
98
99 # ---- structural figure (lambda vs 1/depth), reproducible ----
100 import matplotlib; matplotlib.use("Agg"); import matplotlib.pyplot as _plt
101 _lam=[]; _dep=[]
102 _aapl=aapl
103 for _hh,_g in _aapl.groupby("hh"):
104     if len(_g)<50: continue
105     _lam.append(np.polyfit(_g["o1"],_g["dmid"],1)[0]); _dep.append(_g["dep"].mean())
106 _lam=np.array(_lam); _dep=np.array(_dep)
107 _fig,_ax=_plt.subplots(figsize=(6,4.5))
108 _ax.scatter(1/_dep,_lam,s=30,color="#1f77b4")
109 _c=np.polyfit(1/_dep,_lam,1); _xs=np.linspace((1/_dep).min(),(1/_dep).max(),50)
110 _ax.plot(_xs,np.polyval(_c,_xs),"r-",label=f"corr={np.corrcoef(_lam,1/_dep)[0,1]:.2f}")

```

```

111 _ax.set_xlabel("1 / average best-level depth (1/shares)"); _ax.set_ylabel("Impact lambda (cents/share)"
    )
112 _ax.set_title("Price impact scales inversely with depth"); _ax.legend(); _ax.grid(alpha=.3)
113 import os as _os; _os.makedirs("outputs",exist_ok=True); _fig.tight_layout(); _fig.savefig("outputs/
    fig12_structural.png",dpi=140)

```

References

- [1] Cont, R., Kukanov, A., & Stoikov, S. (2014). The price impact of order book events. *Journal of Financial Econometrics*, 12(1), 47–88.
- [2] Cont, R., Cucuringu, M., & Zhang, C. (2023). Cross-impact of order flow imbalance in equity markets. *Quantitative Finance*, 23(10), 1373–1393.
- [3] Huang, R., & Polak, T. LOBSTER: Limit Order Book System — the efficient reconstructor. Sample data, AAPL 2012-06-21.